SISTEMA INTELIGENTE PARA LA CLASIFICACIÓN DE RESIDUOS RECICLABLES MEDIANTE VISIÓN ARTIFICIAL BAJO LA METODOLOGÍA CRISP-DM

FASE 1 - COMPRENSIÓN DEL NEGOCIO (CRISP-DM)

Problema

La clasificación manual presenta errores humanos.

Pregunta de investigación

¿Es posible desarrollar un sistema inteligente basado en EfficientNetB0?

FASE 2 - COMPRENSIÓN DE LOS DATOS (CRISP-DM)

In [ ]:
```
# ============================================================
# INSTALACION DE LIBRERIAS
# ============================================================
#   - tensorflow      -> crear y entrenar la red neuronal (Deep Learning)
#   - matplotlib      -> dibujar graficas (accuracy, loss, distribucion)
#   - seaborn         -> permite mejor diseño de graficas  (matriz de confusion)
#   - kaggle          -> descargar el dataset TrashNet desde Kaggle
#   - scikit-learn    -> metricas de evaluacion (reporte y matriz de confusion)
!pip install tensorflow matplotlib seaborn kaggle scikit-learn
```

In [ ]:
```
# IMPORTACION DE LIBRERIAS

import os                           # manejar carpetas y rutas de archivos
import numpy as np                  # operaciones con matrices/numeros (arrays)
import pandas as pd                 # manejar tablas de datos (no muy usado aqui)
import matplotlib.pyplot as plt     # graficas basicas
import seaborn as sns               # graficas estadisticas (matriz de confusion)
import tensorflow as tf             # libreria principal de Deep Learning
import zipfile
```

Descargar Dataset

In [ ]:
```
#Configuramos el entorno de descarga
import os
os.environ['KAGGLE_CONFIG_DIR']="C:/Users/ADMIN-ITQ/.kaggle"
```

In [ ]:
```
#Creamos la carpeta dataset
os.makedirs("dataset", exist_ok=True)
```

In [ ]:
```
# CONFIGURAR KAGLE Y DESCARGAR EL DATASET TRASHNET

# !mkdir ~/.kaggle                        # crear la carpeta oculta donde va la llav

# !cp kaggle.json ~/.kaggle/              # copiar la llave subida a esa carpeta

# !chmod 600 ~/.kaggle/kaggle.json        # dar permisos seguros (solo el dueno la l

!kaggle datasets download -d feyzazkefe/trashnet
```

In [ ]:
```
#Descompriminos el archivo
with zipfile.ZipFile("trashnet.zip", 'r') as zip_ref:
    zip_ref.extractall("dataset")
```

In [ ]:
```
# Ruta (direccion de la carpeta) donde quedaron las imagenes ya descomprimidas.
ruta_datos = "dataset/dataset-resized"
```

In [ ]:
```
# Listar las subcarpetas (clases) que hay dentro del dataset.

lista_datos=os.listdir(ruta_datos)
print(lista_datos)
```

In [ ]:
```
# CONTAR CUANTAS IMAGENES HAY POR CLASE

nombre_clases = []     # lista vacia para guardar los nombres de las clases
cantidad = []   # lista vacia: para guardar cuantas imagenes tiene cada clase

# Recorremos una por una las carpetas que hay dentro de ruta_datos
for clase in lista_datos:

    ruta_clase = os.path.join(ruta_datos, clase)

    cant_img_clase = len(os.listdir(ruta_clase))

    nombre_clases.append(clase)

    cantidad.append(cant_img_clase)

total_imagenes = sum(cantidad)

print(f"{len(nombre_clases)} clases")
print(f"{total_imagenes} imagenes")
```

```
# Utilizar DataFrame de pandas para ver clase y cantidad ordenado
tabla = pd.DataFrame({
    "Clase": nombre_clases,
    "Cantidad": cantidad
})

tabla
```

In [ ]:
```
# GRAFICA DE BARRAS: DISTRIBUCION DEL DATASET

plt.figure(figsize=(10, 6))          # crea el lienzo de la grafica con tamano 10x6

plt.bar(nombre_clases, cantidad)            # dibuja barras: eje X = clases, altura

plt.title("Distribución del Dataset")  # titulo de la grafica

plt.xlabel("Clase")                  # etiqueta del eje X (horizontal): nombre de l

plt.ylabel("Cantidad")               # etiqueta del eje Y (vertical): numero de ima

plt.show()                           # muestra la grafica en pantalla
```

FASE 3 - PREPARACIÓN DE LOS DATOS

In [ ]:
```
# PARAMETROS DE LAS IMAGENES

tamano_imagen = (224, 224)

tamano_lote = 32   # define el numero de imagenes que se procesan juntas en cada pa
                   # (un "lote"; ayuda a entrenar mas rapido y de forma e
```

In [ ]:
```
# CARGAR IMAGENES DE ENTRENAMIENTO (70%)

# Lee las imagenes desde las carpetas y las convierte en un dataset de TensorFlow.
# Keras asigna automaticamente la etiqueta segun el nombre de la carpeta.

datos_entrenamiento = tf.keras.utils.image_dataset_from_directory(  # funcion de Ke

    ruta_datos,              # carpeta que contiene las subcarpetas por clase

    validation_split=0.30,   # 'validation_split' (parametro de Keras): reserva un

    subset="training",       # 'subset' (parametro de Keras): pedimos el 70% de ENT

    seed=123,                # 'seed' = semilla fija: hace que el reparto sea siemp

    image_size=tamano_imagen,   # 'image_size': redimensiona cada imagen a 224x224

    batch_size=tamano_lote      # 'batch_size': agrupa las imagenes en lotes de 32

)
```

In [ ]:
```
# CARGAR EL 30% RESTANTE (se dividira luego en validacion + prueba)

# Usa la MISMA semilla (123) que el entrenamiento para no mezclar imagenes entre co

datos_temporales = tf.keras.utils.image_dataset_from_directory(  # mismo cargador a

    ruta_datos,              # misma carpeta de imagenes

    validation_split=0.30,   # mismo 30% reservado

    subset="validation",     # 'subset="validation"': ahora pedimos ese 30% restant

    seed=123,                # MISMA semilla -> el reparto es consistente con el de

    image_size=tamano_imagen,   # mismo tamano 224x224

    batch_size=tamano_lote      # mismos lotes de 32

)
```

In [ ]:
```
# ============================================================
# DIVIDIR EL 30% EN: PRUEBA (15%) + VALIDACION (15%)
# ============================================================
# datos_temporales (30%) se parte en dos mitades iguales:
#   - datos_prueba      -> para la evaluacion FINAL (datos que el modelo nunca vio)
#   - datos_validacion  -> para vigilar el entrenamiento epoca a epoca

lotes_validacion = tf.data.experimental.cardinality(datos_temporales)  # 'cardinali

datos_prueba = datos_temporales.take(lotes_validacion // 2)   # 'take' = toma la PR

datos_validacion = datos_temporales.skip(lotes_validacion // 2)  # 'skip' = salta e
                                                                  # ('//' es divisi
```

In [ ]:
```
# ============================================================
# OPTIMIZACION DE DATOS

#Normalización
# EfficientNetB0 YA normaliza las imagenes por dentro y espera los pixeles
# en el rango [0, 255]. Si se divide entre 255  el rango se convierte en [0, 1] y

# 1) ordenamos  las clases en orden alfabetico ANTES de transformar los datos   est
print("Orden REAL de las clases:", nombre_clases)

# 2) Optimizacion de rendimiento (se utiliza cache + prefetch = entrena mas rapido)
#    NO cambia los datos, solo acelera la lectura desde el disco.
autotune = tf.data.AUTOTUNE   # 'AUTOTUNE' = valor de TensorFlow que ajusta solo la

datos_entrenamiento = datos_entrenamiento.cache().prefetch(autotune)  # 'cache' gua
datos_validacion    = datos_validacion.cache().prefetch(autotune)
datos_prueba        = datos_prueba.cache().prefetch(autotune)
```

FASE 4 - MODELADO

In [ ]:
```
# ============================================================
# CARGAR EL MODELO BASE: EfficientNetB0 (TRANSFER LEARNING)
# ============================================================
# En vez de entrenar una red desde cero (necesitaria millones de imagenes),
# reutilizamos EfficientNetB0 que YA aprendio a "ver" con ImageNet.
from tensorflow.keras.applications import EfficientNetB0   # importamos el modelo E

modelo_base = EfficientNetB0(        # creamos el modelo base

    weights='imagenet',          # 'weights="imagenet"': trae los pesos ya entrenad

    include_top=False,           # 'include_top=False': quitamos la ultima capa ori
                                 # para poner la nuestra (6 clases de residuos)

    input_shape=(224, 224, 3)    # 'input_shape': tamano de entrada = 224x224 pixel

)

modelo_base.trainable = False    # '.trainable = False': CONGELAMOS el modelo base.
                                 # No se reentrena; solo aprovechamos lo que ya sab
```

In [ ]:
```
from tensorflow.keras.applications.efficientnet import preprocess_input

# CONSTRUIR EL MODELO


aumento_datos = tf.keras.Sequential([                                # 'Sequential'
    tf.keras.layers.RandomFlip("horizontal"),                        # voltea la im
    tf.keras.layers.RandomRotation(0.15),                            # la rota un p
    tf.keras.layers.RandomZoom(0.15),                                # le hace zoom
    tf.keras.layers.RandomTranslation(0.1, 0.1),                     # la mueve (el
    tf.keras.layers.RandomContrast(0.2),                             # varia el con
    tf.keras.layers.RandomBrightness(0.2, value_range=(0, 255)),     # varia el bri
], name="aumento_de_datos")                                          # 'name': nomb

modelo = tf.keras.Sequential([                   # creamos el modelo final apilando

    tf.keras.layers.Input(shape=(224, 224, 3)),  # 'Input': capa de entrada, recibe

     #  aplica las variaciones (solo durante el entrenamiento)
    aumento_datos,

    #Normalizo los datos para las imagenes
    tf.keras.layers.Lambda(preprocess_input),

    modelo_base,                                 #  EfficientNetB0 congelado (extra

    tf.keras.layers.GlobalAveragePooling2D(),    # 'GlobalAveragePooling2D': resume

    tf.keras.layers.Dense(256, activation='relu'),  # 'Dense(128)': capa de 128 neu
                                                    # 'relu' = funcion de activaci

    tf.keras.layers.Dropout(0.4),                # 'Dropout(0.3)': apaga el 30% de
```

```
    tf.keras.layers.Dense(len(nombre_clases), activation='softmax')  # capa FINAL:
                                                  # 'softmax' = convierte las salida

])

modelo.summary()   # '.summary()': imprime un resumen de las capas y cuantos parame
```

In [ ]:
```
# ============================================================
# COMPILAR EL MODELO (configurar como va a aprender)
# ============================================================
modelo.compile(   # '.compile()': prepara el modelo para entrenar (define como apre

    optimizer='adam',                          # 'optimizer="adam"': algoritmo que

    loss='sparse_categorical_crossentropy',    # 'loss': funcion de error para clas
                                               # "sparse" porque las etiquetas son

    metrics=['accuracy']                       # 'metrics=["accuracy"]': la metrica

)
```

In [ ]:
```
# ENTRENAR EL MODELO

# Una EPOCA = una pasada completa por todas las imagenes de entrenamiento.


conteo = {c: len(os.listdir(os.path.join(ruta_datos, c))) for c in nombre_clases}
suma_imagenes = sum(conteo.values())
peso_clases = {i: suma_imagenes / (len(nombre_clases) * conteo[c])
               for i, c in enumerate(nombre_clases)}
print("Peso de cada clase:", peso_clases)
#  CALLBACKS: ayudas automaticas que actuan durante el entrenamiento.
controles = [                                  # 'controles' = nuestra lista de ayu

    tf.keras.callbacks.EarlyStopping(          # 'EarlyStopping': detiene el entren
        monitor='val_accuracy',                # 'monitor': vigila la exactitud en
        patience=5,                            # 'patience=5': espera 5 epocas sin
        restore_best_weights=True,             # 'restore_best_weights': se queda c
        verbose=1),                            # 'verbose=1': avisa por pantalla cu

    tf.keras.callbacks.ReduceLROnPlateau(      # 'ReduceLROnPlateau': baja la tasa
        monitor='val_loss',                    # vigila el error en validacion
        factor=0.5,                            # 'factor=0.5': cuando se estanca, r
        patience=3,                            # espera 3 epocas sin mejora antes d
        verbose=1),                            # avisa por pantalla cuando actua
]

# 3) ENTRENAR
historial = modelo.fit(            # '.fit()': inicia el entrenamiento; 'historial'
    datos_entrenamiento,           # datos con los que el modelo APRENDE
    validation_data=datos_validacion,  # 'validation_data': datos con los que se au
    epochs=25,                     # 'epochs=25': maximo 25 pasadas (EarlyStopping
    class_weight=peso_clases,      # 'class_weight': aplica los pesos para compensa
    callbacks=controles            # 'callbacks': nuestra lista de ayudas (EarlySto
```

```
)
# 'historial' guarda el accuracy y loss de cada epoca para graficarlos despues.
```

In [ ]:
```
# GRAFICA DE PRECISION (ACCURACY)

# Compara como mejora el % de aciertos en ENTRENAMIENTO vs VALIDACION.
# - Si ambas suben juntas -> el modelo esta aprendiendo bien.
# - Si entrenamiento sube mucho y validacion se queda abajo -> sobreajuste.

plt.figure(figsize=(10, 6))   # crea el lienzo de la grafica (tamano 10x6)

plt.plot(historial.history['accuracy'],     marker='o', label='Entrenamiento')  # l
plt.plot(historial.history['val_accuracy'], marker='o', label='Validación')     # l
# 'historial.history' = diccionario con los numeros guardados de cada epoca

plt.title('Precisión (Accuracy) por época', fontsize=14, fontweight='bold')  # titu
plt.xlabel('Época')                  # eje X: numero de epoca
plt.ylabel('Precisión (0 a 1)')      # eje Y: exactitud (0 = 0%, 1 = 100%)
plt.ylim(0, 1)                       # 'ylim': fija el eje Y de 0 a 1 para leerlo f
plt.legend(loc='lower right')        # 'legend': muestra el cuadrito que dice que l
plt.grid(True, alpha=0.3)            # 'grid': dibuja una cuadricula suave de fondo
plt.show()                           # muestra la grafica
```

In [ ]:
```
# GRAFICA DE PERDIDA (LOSS)

# El "loss" (error) mide que tan equivocado esta el modelo. DEBE BAJAR con las epoc
# - Si el loss de validacion empieza a SUBIR mientras el de entrenamiento baja
#   -> el modelo se esta sobreajustando (memoriza en vez de aprender).

plt.figure(figsize=(10, 6))   # crea el lienzo de la grafica

plt.plot(historial.history['loss'],     marker='o', label='Entrenamiento')  # linea
plt.plot(historial.history['val_loss'], marker='o', label='Validación')     # linea

plt.title('Pérdida (Loss) por época', fontsize=14, fontweight='bold')  # titulo
plt.xlabel('Época')                  # eje X: numero de epoca
plt.ylabel('Pérdida (error)')        # eje Y: valor del error (mas bajo = mejor)
plt.legend(loc='upper right')        # cuadrito de leyenda
plt.grid(True, alpha=0.3)            # cuadricula suave de fondo
plt.show()                           # muestra la grafica
```

In [ ]:
```
# AJUSTE FINO
#fineTunnig


# (1e-5 = 0.00001). Esto ajusta finamente la red al dataset de residuos y suele
# subir la precision varios puntos.

modelo_base.trainable = True   # '.trainable = True': DESCONGELAMOS el modelo base


for capa in modelo_base.layers:                                  # recorremos cada capa
    if isinstance(capa, tf.keras.layers.BatchNormalization): # 'isinstance' = pregu
        capa.trainable = False                                   # si lo es, la dejamos
```

```
# Recompilar SIEMPRE despues de cambiar 'trainable', con tasa MUY baja
# para no destruir lo que el modelo ya aprendio.
modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),  # 'Adam' con 'learning
    loss='sparse_categorical_crossentropy',                  # misma funcion de err
    metrics=['accuracy']                                     # misma metrica: exact
)

historial_ajuste = modelo.fit(          # entrenamos otra vez; guardamos el resultad
    datos_entrenamiento,                # mismos datos de entrenamiento
    validation_data=datos_validacion,   # mismos datos de validacion
    epochs=10,                          # solo 10 epocas extra (es un ajuste fino)
    class_weight=peso_clases,           # seguimos compensando el desbalance
    callbacks=controles                 # reutilizamos las mismas ayudas (EarlyStopp
)
```

FASE 5 - EVALUACIÓN

In [ ]:
```
#Verificacion de predicciones
print("="*60)
print("VERIFICACIÓN DE PREDICCIONES")
print("="*60)
for imagenes, etiquetas in datos_prueba.take(1):
    predicciones = modelo.predict(imagenes)
    confianza = np.max(predicciones, axis=1)
    clases_predichas = np.argmax(predicciones, axis=1)
    for i in range(min(10, len(etiquetas))):

        clase_real = nombre_clases[int(etiquetas[i])]
        clase_predicha = nombre_clases[clases_predichas[i]]

        print(f"Clase Real     : {clase_real}")
        print(f"Clase Predicha : {clase_predicha}")
        print(f"Confianza      : {confianza[i]*100:.2f}%")

        print("-"*50)
```

In [ ]:
```
# HERRAMIENTAS DE EVALUACION (scikit-learn)

from sklearn.metrics import classification_report   # reporte: precision, recall, f

from sklearn.metrics import confusion_matrix         # tabla de aciertos vs confusi
```

In [ ]:
```
# ============================================================
# EVALUACION FINAL CON DATOS DE PRUEBA (datos_prueba)
# ============================================================
# Mide el rendimiento REAL con imagenes que el modelo NUNCA vio al entrenar.
# Este es el numero clave para defender el proyecto.

perdida_prueba, exactitud_prueba = modelo.evaluate(datos_prueba)  # '.evaluate()' a
```

```
print(f"\nPrecisión final en prueba: {exactitud_prueba*100:.2f}%")  # mostramos la
print(f"Pérdida final en prueba:   {perdida_prueba:.4f}")          # mostramos el
```

In [ ]:
```
# ============================================================
# OBTENER PREDICCIONES PARA EL REPORTE Y LA MATRIZ
# ============================================================
# Recorre el conjunto de prueba y guarda:
#   - etiquetas_reales:   la clase REAL de cada imagen
#   - etiquetas_predichas: la clase que PREDIJO el modelo

etiquetas_reales = []      # lista vacia para las clases verdaderas
etiquetas_predichas = []   # lista vacia para las clases que adivina el modelo

for imagenes, etiquetas in datos_prueba:           # recorremos los lotes de prueba

    prediccion = modelo.predict(imagenes)          # '.predict()' = el modelo da la

    etiquetas_reales.extend(etiquetas.numpy())     # 'extend' agrega las etiquetas

    etiquetas_predichas.extend(np.argmax(prediccion, axis=1))  # 'np.argmax' = elig
```

In [ ]:
```
# ============================================================
# REPORTE DE CLASIFICACION
# ============================================================
# Por cada clase muestra:
#   - precision: de lo que predijo como X, cuanto era realmente X
#   - recall:    de todo lo que era X, cuanto detecto
#   - f1-score:  equilibrio entre precision y recall (1.0 = perfecto)
print(classification_report(   # 'classification_report' (de scikit-learn) arma la
    etiquetas_reales,          # clases verdaderas
    etiquetas_predichas,       # clases que predijo el modelo
    target_names=nombre_clases,  # 'target_names': muestra el NOMBRE de cada clase
    zero_division=0            # 'zero_division=0': evita avisos si alguna clase no
))
```

In [ ]:
```
# ============================================================
# MATRIZ DE CONFUSION
# ============================================================
# Tabla: filas = clase REAL, columnas = clase PREDICHA.
# La DIAGONAL (esquina sup-izq a inf-der) son los ACIERTOS.
# Todo lo de fuera de la diagonal son CONFUSIONES (errores).
# Ej: cuantas veces una "glass" real la predijo como "plastic".

matriz = confusion_matrix(etiquetas_reales, etiquetas_predichas)  # 'confusion_matr

plt.figure(figsize=(9, 7))   # lienzo de la grafica
sns.heatmap(                 # 'heatmap' (de seaborn) dibuja la tabla con colores
    matriz,                  # la matriz de numeros a dibujar
    annot=True,              # 'annot=True': escribe el numero dentro de cada celda
    fmt='d',                 # 'fmt="d"': formato de numero entero (sin decimales)
    cmap='Blues',            # 'cmap="Blues"': escala de color azul (mas oscuro = m
    xticklabels=nombre_clases,  # nombres de clase en el eje X (orden REAL del mode
    yticklabels=nombre_clases   # nombres de clase en el eje Y
)
plt.title('Matriz de Confusión', fontsize=14, fontweight='bold')  # titulo
```

```
plt.xlabel('Predicción del modelo')  # eje X: lo que el modelo predijo
plt.ylabel('Clase real')             # eje Y: lo que realmente era
plt.tight_layout()                   # 'tight_layout': acomoda todo para que no se
plt.show()                           # muestra la grafica
```

FASE 6 - DESPLIEGUE

In [ ]:
```
# ============================================================
# GUARDAR EL MODELO ENTRENADO
# Guarda toda la red (arquitectura + pesos aprendidos) en un solo archivo.
# Asi se puede reutilizar despues SIN volver a entrenar (ej. en la API con FastAPI)
modelo.save("modelo_residuos.keras")   # '.save()': guarda el modelo en el archivo
```

In [ ]:
```
# ============================================================
# DESCARGAR EL MODELO A TU COMPUTADORA
# ============================================================
# Baja el archivo .keras desde Colab a tu PC para usarlo en el proyecto local
# (la API de FastAPI/Flask lo cargara para hacer las predicciones).
from google.colab import files

files.download("modelo_residuos.keras")
```

Clasificación de imágenes desde la cámara

Ahora, vamos a añadir la funcionalidad para clasificar imágenes capturadas directamente desde tu cámara web usando el modelo entrenado. Necesitaremos cargar el modelo guardado, capturar una imagen, preprocesarla para que coincida con la entrada del modelo y luego realizar la predicción.

In [ ]:
```
from tensorflow.keras.preprocessing import image
import numpy as np

def predecir_seguro(ruta_imagen):

    img = image.load_img(
        ruta_imagen,
        target_size=(224,224)
    )

    img_array = image.img_to_array(img)

    img_array = np.expand_dims(
        img_array,
        axis=0
    )

    predicciones = modelo.predict(
        img_array,
        verbose=0
    )
```

```
    indice = np.argmax(predicciones)

    confianza = float(
        np.max(predicciones)
    )

    if confianza < 0.75:

        return {
            "clase":"desconocido",
            "confianza":confianza
        }

    return {
        "clase":nombre_clases[indice],
        "confianza":confianza
    }
```

In [ ]:
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("imagenes_reales/botella1.jpeg")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.show()

print(img.shape)
```

In [ ]:
```
crop = img[400:1300, 390:700]
crop_rgb = cv2.cvtColor(crop, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6,6))
plt.imshow(crop_rgb)
plt.axis("off")
plt.show()
```

In [ ]:
```
cv2.imwrite(
    "imagenes_reales/botella_crop.jpg",
    crop
)
```

In [ ]:
```
predecir_imagen(
    "imagenes_reales/botella_crop.jpg"
)
```

In [ ]:
```
import os
print(os.getcwd())
```

In [ ]:
```
from funciones import *
mostrar_segmentacion(
    "imagenes_reales/botella1.jpeg"
)
```